SANR-CMF: Semantic-Aware Naming Resolution and Cache Management Framework in Information-Centric Network for Internet of Things

Ghulam Musa Raza, Ihsan Ullah, Muhammad Nadeem Ali, and Byung-Seo Kim, *Senior Member, IEEE*

***Abstract*—In-network caching stands as one of the core pillars of the named data networking (NDN) paradigm, where every internet router has the capability to store the incoming data packets in its cache along the path between data sources and consumers. This capability is particularly vital in Internet of Things (IoT) environments, where efficient and timely data retrieval is critical. Numerous strategies have been designed for caching IoT data and often rely on the exact match policy (EMP), which strictly matches content names for data retrieval. However, semantic considerations of content names in EMP remain underexplored. Additionally, several caching mechanisms are modeled after traditional IP-based communication approaches, such as neighbor or topology-based centrality methods for content placement and data eviction for replacement. These approaches hold limited relevance in NDN, as content names can originate from anywhere in the network. To this end, we propose a novel semantic-aware naming resolution as an alternative to EMP in cache design. Furthermore, the proposed scheme introduces a content-based centrality mechanism, along with a backup strategy for cache management. Integration of semantics into caching strategies offers a more optimal approach for supporting NDN IoT. This combination can boost overall network performance. Simulation results illustrate that the proposed scheme outperformed the advanced existing studies CaDaCa and PoolCache in cache hit ratios, semantic satisfaction rate, latency, and route elongation factor.**

***Index Terms*—Caching, deep neural network (DNN), graph neural network (GNN), Internet of Things (IoT), named data network (NDN), natural language processing (NLP), semantic communication.**

I. Introduction

THE RAPID advancements in wireless and embedded technologies have led to a substantial increase in internet data over the span of the past few years. The rapid increase in sensing and computing devices has led to the development of diverse Internet of Things (IoT) applications, such as smart homes, healthcare systems, and industrial automation [1]. As the number of internet-connected devices grows, people increasingly access information in the form of videos, audio, and text, leading to substantial internet traffic. The growing use of these devices brings several challenges that the current IP-based internet struggles to handle. Such as mobility support is limited as IP addresses are tied to network locations, making seamless handovers complex. Storage capacity issues arise due to the lack of in-network caching, leading to redundant data transfers. Data retrieval efficiency suffers because IP routing relies on host-based addressing rather than content-based access. Latency increases due to multihop transmission and reliance on centralized servers. For the seamless deployment of IoT, a new communication paradigm must be explored. information-centric networking (ICN) is a suggested framework for the future internet that emphasizes content-based communication over host-based communication [2]. Named data networking (NDN), a prominent example of ICN, has garnered significant attention from the community that is pursuing to redesign the internet architecture to address the limitations of the existing transmission control protocol/internet protocol (TCP/IP) model, which is host-centric and not well-suited for today's content-driven usage patterns.

To address the aforementioned limitations of TCP/IP protocols, NDN shifts to a content-centric paradigm, where data is retrieved based on named content rather than host locations. This eliminates the dependency on fixed addresses, enabling seamless mobility and reducing reliance on centralized servers. Additionally, NDN's in-network caching significantly enhances efficiency by allowing content to be retrieved from intermediate nodes, reducing latency and bandwidth consumption. Security is also inherently improved, as NDN authenticates data itself rather than relying on secure host-to-host connections. These features make NDN a more scalable, efficient, and resilient architecture for modern IoT and data-driven applications [3]. To retrieve content, the consumer sends an interest packet containing the content's name, which is then routed through the network. When the requested content is located, it is returned to the consumer along the reverse route taken by the data packet. NDN presents a promising architecture for IoT implementation, offering solutions to key challenges, such as latency, security, scalability, mobility, content-based retrieval, and adaptation.

One of the key strengths of NDN is in-network cache management, where content names are matched using the exact match policy (EMP) algorithm [4]. EMP in the content store (CS) requires a character-by-character complete match in the content name for caching. This implies that content

Received 29 March 2025; revised 22 May 2025; accepted 7 June 2025. Date of publication 11 June 2025; date of current version 25 August 2025. This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) under Grant 2022R1A2C1003549. *(Corresponding author: Byung-Seo Kim.)*

The authors are with the Department of Software and Communications Engineering, Hongik University, Sejong 30016, South Korea (e-mail: ghulammusaraza96@gmail.com; danish1852@gmail.com; nadeem@mail.hongik.ac.kr; jsnbs@hongik.ac.kr).

Digital Object Identifier 10.1109/JIOT.2025.3578699

2327-4662 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.
See https://www.ieee.org/publications/rights/index.html for more information.
Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

names with exactly similar and hierarchical nature to those previously requested and stored in the CS will be considered for caching and subsequent retrieval. IoT-based names are typically transient, fleeting, new, with diverse priorities, highly dynamic, and of different lengths. These names also vary in length and priority, and follow heterogeneous naming conventions–that is, they differ in structure, format, and semantics–due to the diverse communication protocols and data formats used by heterogeneous IoT devices [5]. Therefore, a high likelihood exists that the two content names for IoT networks share the same underlying intent. However, they may differ slightly or significantly in their naming conventions. SANR-CMF assumes a scenario of a smart campus of hongik university where two requests with content names, such as "**samrt%**hongikuni**/versity/**Sejong/bcn**/prof**kim/info" and "hongikuni/Sejong/bcn**lab/dr**kim/info" arrive.

In the provided example, both content names share the same intent; however, they fail to exactly match. The first content name uses the full **"smart**%hongikuni**versity**," whereas the second uses the abbreviated "hongik**uni**." Additionally, "**prof**kim" refers to professor Kim in the first name, whereas "**dr**kim" is used in the second. The first name uses the lab's name directly "**bcn**," while the second mentions "bcn**lab**." Therefore, the EMP fails to work. As a result, the CS will be ineffective in retrieving the data and the request will be forwarded to the forwarding information base (FIB). The rationale behind this lies in name ambiguity which occurred in both NDN IoT content names. Therefore, the naming resolution for matching content names for NDN IoT is still an open challenge.

Additionally, NDN IoT content typically has a finite lifespan, after which it is deemed outdated. This feature of IoT introduces complexity to caching IoT data compared to caching traditional internet data, where content may have a longer lifespan or be static. As a result, what IoT data to cache or discard in the cache nodes is still an open question [6]. To this end, traditional schemes follow caching metrics, including repetition, popularity, freshness, etc., with EMP's backend support. There is a possibility that numerous requests for the same content exist, however, due to naming ambiguity, there are no cache hits for the corresponding content [4], [7]. This is justified by the fact that traditional metrics overlook the semantic relevance of content names, which can be critical in ensuring that frequently needed data is efficiently cached. Therefore, semantic incorporation with content selection metrics is required as well. This correlation can focus on understanding the meaning and context of the content names, which can mitigate issues related to name ambiguity and improve cache performance.

Following the semantic selection of content for caching, the notable challenge is determining the optimal placement. To this end, content is stored on suitable nodes called caching nodes. This feature provides several advantages, including simplifying and accelerating data retrieval for subscribers, reducing request traffic near IoT data publishers, and decreasing the energy consumption of IoT data sources [8]. By leveraging the concept of cache nodes, a strategy commonly employed by state-of-the-art methodologies is to place the content on a base of a node or topology centrality. However, in NDN, requesters and producers can be located anywhere in the network, potentially far from each other or centrality-based cache nodes. In a cache-supported network, which may encompass numerous nodes constantly entering and leaving the area of interest packet due to mobility feature of IoT devices, achieving efficient cache node selection is crucial to support large-scale and dynamic caching operations [9]. Cache nodes do not need to always be nodes with high neighbor counts or dependent on network topology. Therefore, selecting the specific node where semantic content is cached, is critical in NDN as it directly impacts the overall performance of the network.

In addressing the aforementioned issues, this article proposes a natural language processing (NLP) [10] supported semantic-based naming resolution for the content names in the CS table. To this end, the proposed scheme leverages and modifies the mechanisms of NLP. Two content names will be similar if both share the same semantic intent regardless of the spelling difference. Afterward, the proposed scheme selects semantically popular content instead of normal popular content. Finally, the proposed scheme selects the cache nodes on the basis of content centrality regardless of node or topology centrality. Those nodes are selected as cache nodes where requests for corresponding content are coming repeatedly. The major contributions of the proposed scheme are as follows.

1) We propose a semantic aware naming resolution and cache management framework, called SANR-CMF. The SANR-CMF eliminates name ambiguities that may arise in the EMP of the CS table. By leveraging the hierarchical structure and vocabulary of NDN content names, SANR-CMF proposes a novel NLP-based strategy to semantically match content names, thereby enhancing cache hits and facilitating an efficient semantic satisfaction rate (SSR).
2) By examining the semantic nature of content names, SANR-CMF proposes the selection of the most semantically repeated requests (MSRR) content rather than merely the most frequently requested content.
3) SANR-CMF offers semantic-supported Content centrality-based nodes (CCN) in NDN IoT for content placement. Specifically, MSRRs are placed exclusively on those nodes (CCN) where they are in high demand.
4) Additionally, each NDN content in CCN has a timestamp in the CS. When the timestamp expires or the cache is full to accommodate incoming content, the content with the least semantic resemblance is removed from the CCN to make space for new content. In this way, SANR-CMF introduces the capability to remove and store content at backup nodes as part of the content replacement strategy.

The sections of this article are structured as follows. Section II provides some detailed background on NDN and state-of-the-art relevant works. Motivation for SANR-CMF is discussed in Section III. Section IV specifies the proposed methodology and use case. Section V explains the simulation study and evaluation of results. Finally, Section VI concludes our work and highlights the future direction.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

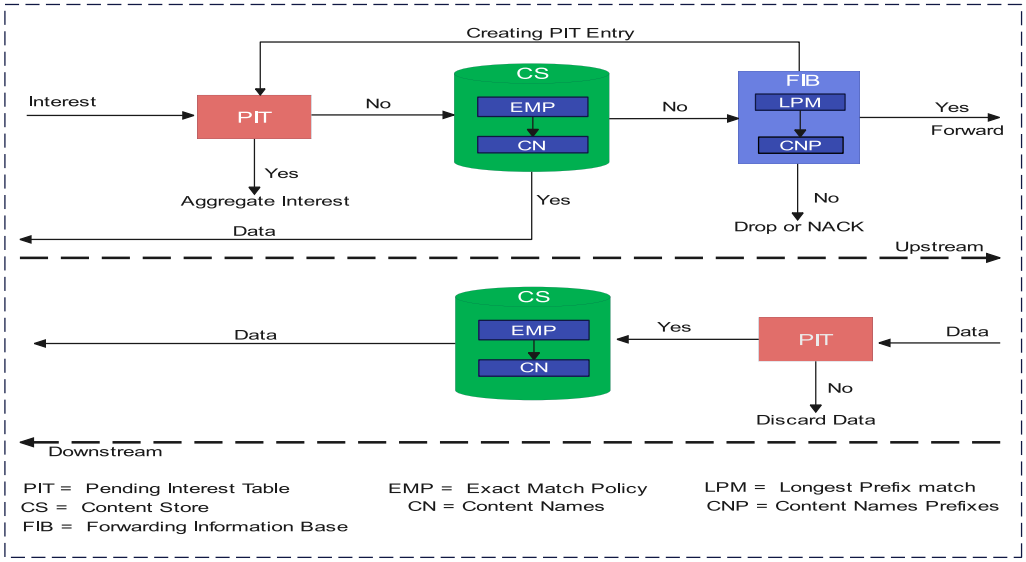


Fig. 1. NDN communication process.

II. Background and Related Work

A. NDN in a Nutshell

NDN is a future internet architecture that shifts the focus from addressing locations (IP addresses) to addressing the content by names. Unlike the traditional IP-based internet, where communication is established through endpoints, NDN facilitates data retrieval based on content names, enhancing security, scalability, and efficiency. In NDN, three main data structures are pivotal: 1) the pending interest table (PIT); 2) the CS; and 3) the FIB, as presented in Fig. 1.

The PIT keeps track of interest packets that have been forwarded and are awaiting corresponding data packets, ensuring efficient packet delivery. The CS acts as a cache that temporarily stores received data packets to satisfy future interest packets without further forwarding again. Regarding content name matching, the CS uses an EMP to determine if the data against the corresponding interest packet is already in the cache. When an interest packet arrives, the CS is checked to confirm the exact name matches with already stored content name. If a match is found, the corresponding data is retrieved and sent back to satisfy the interest packet, avoiding the need to forward the interest packet further. If no match is found, the interest packet is forwarded to FIB. The FIB is used for forwarding interest packets based on the longest prefix match (LPM), similar to routing in traditional IP networks, however, it operates on content names rather than IP addresses. The FIB stores the content name prefixes with corresponding interfaces. For forwarding decisions, FIB follows the LPM, which determines the best matching route for an incoming interest packet.

B. Related Work

NDN caching serves as a valuable strategy for enhancing data transmission services in different IoT contexts [18]. Therefore, numerous scholarly works have explored caching strategies within ICN and NDN in recent years. For instance, the standard caching approach in NDN IoT, called cache everything everywhere (CEE) [19], presents a basic solution that enhances the accessibility and availability of data. Nevertheless, CEE may lead to significant content duplication, resulting in higher bandwidth consumption and energy expenditure. Moreover, caching data across all network nodes can create unnecessary cache allocation and inefficient utilization of resources. The client cache strategy (CCS) [11] enhances content accuracy by assessing its popularity according to the volume of requests received at individual network nodes. While effective, this approach can be resource-intensive and may affect caching performance, particularly in retrieval speed and cache hit efficiency.

Other advanced strategies [12], [20] are designed for NDN IoT to address different aspects of caching efficiency. Tag-based caching, derived from popularity-based caching, aims to boost the forwarding performance of content by employing tag filters. In contrast, the sleep-driven caching approach focuses on enhancing the energy efficiency of NDN-based IoT environments by optimizing the energy consumption of network nodes. Leave copy down (LCD) [13] was designed to improve the content caching architecture by minimizing data redundancy. However, it often leads to a rise in the duplication of similar data. To address the challenges posed by LCD and CEE strategies, moving copy down (MCD) [21] was presented to eliminate redundant data along the data delivery route. Additionally, the edge caching strategy (ECS) [14] intends to minimize the length of the route between the consumer and the stored content thus better meeting end-user demands. The goal is to minimize content fetching time for later interest packets. Probcache [15] approximates the caching capability of a path and probabilistically caches content to reserve cache space for other flows that share portions of the same path, and fairly distribute the content of different flows across caches along the shared path. The authors estimate the probability of caching specific content by taking into account the number of hops from the content producer to the consumer, as well as the remaining hops along the route to the consumer.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF CACHING STRATEGIES IN NDN/IOT LITERATURE

Paper	Challenge Considered	Proposed Solution	Semantic	Cache Placement	Replacement Policy	Future Directions
[11]	High energy use from redundant caching and lack of collaboration.	Collaborative caching using GA with on-/off-path strategy.	No	Hybrid on-/off-path caching	GA-based replacement	Popularity prediction, multi-request support, blockchain security
[12]	Naming inconsistency affects interest expression.	Tag-assisted CCN with Counting Bloom Filters.	No	Tag-based placement using popularity thresholds	Tag-threshold based	Tag vocabulary, ambiguity handling, popularity analysis
[13]	Redundancy and poor diversity due to uniform replication.	EHCP with dual placement at central and edge nodes.	No	Popular content at central, others at edge	LRU	Improve caching performance under topological and memory constraints
[14]	Edge-only caching lacks popularity/freshness awareness.	CES + CCS using popularity and freshness metrics.	No	CES: dynamic popularity; CCS: probabilistic	CES: utility index; CCS: LRU	Add topological centrality and SDN coordination
[15]	Redundant and inefficient decentralized caching.	Probabilistic caching via flow distance.	No	Distance- and capacity-based probability	LRU	Optimize resource use in heterogeneous environments
[16]	Popularity estimation inefficient with unbounded name-based IDs.	Category-based probabilistic caching.	Partial	Threshold by semantic category popularity	LRU/LFU per category	Extend categorization to PIT/FIB; deploy ML on nodes
[17]	Redundant storage and low cache diversity.	Cluster-based collaborative caching.	No	One node per cluster via hash mapping	LRU	Lightweight, popularity-aware coordination

A notable contribution and our target article is the categorized data for caching (CaDaCa) [16] approach, which introduces a strategy by categorizing content names rather than tracking each name individually. The authors employ a multinomial naive bayes (MNB) classifier to group content requests into categories based on their naming hierarchy and past history. This classification allows for more efficient cache management by leveraging common hierarchical structures (e.g., prefixes) found in content names. The placement and replacement strategies in CaDaCa rely on category popularity, ensuring that popular categories are cached more frequently and less popular content is evicted during cache replacement. CaDaCa categorizes content based on historical data, which may show delayed adaptation to dynamic changes in content trends or emerging categories. CaDaCa is merely suitable only for small-scale networks where data can be arranged in categorization. Every node is allocated a distinct data portion based on the content category. However, when it comes to cache node selection, CaDaCa overlooks the critical importance of optimizing which nodes should cache the data. In addition, if content is misclassified into an incorrect category, it could lead to inefficient caching decisions, with popular content being evicted prematurely or less relevant content being cached. Furthermore, the MNB assumes that the features (n-grams) are independent of each other, which is rarely the case in real-world scenarios. This oversimplified assumption can lead to suboptimal categorization, especially for content names where relationships between words (context) are crucial. Another base article for the proposed scheme [17] reduces cache redundancy and enhances content diversity by proposing a node centrality-based collaborative caching mechanism, PoolCache. The authors designed the scheme to enhance the overall caching efficiency within predefined clusters of specific nodes. Their algorithm eliminates content redundancy within each cluster, thereby promoting a more diverse distribution of cached content across the network. However, PoolCache fails to explicitly consider the popularity of incoming content, potentially leading to the unnecessary caching of less frequently requested data. Additionally, PoolCache is a hash function-based cache allocation that lacks dynamic adaptability and increases retrieval latency in case of cache misses, making it less optimal for highly dynamic and large-scale NDN IoT environments. The technical key points from the literature discussed above are presented in Table I.

C. NLP Integration With NDN

NLP has gained significant attention from researchers, its application within NDN remains relatively unexplored. A notable study, semantic information-centric network (SICN) [22] integrates a naming scheme with forwarding mechanisms using NLP. The SICN framework is one of the few that considers the semantic aspects of ICN. It proposes that both publishers and subscribers hold dynamic addresses that can adapt based on their geographic position within the network. Additionally, the naming system for these dynamic addresses should explicitly convey the nature of the information they represent. To achieve this, SICN introduces a 3-D naming system. Users (publishers/subscribers) are required to specify at least one of three dimensions: publisher ID, semantic ID, and geographical ID. The SICN forwarding algorithm employs three tables that encompass these address dimensions. The semantic ID table links the semantic address to the publisher ID, while the geographical ID table connects the publisher ID to the geographical address. The geographical-semantic table correlates the semantic and geographical addresses. SICN routers use these tables for interest packet forwarding, broadcasting the packet if no match is found. SICN lacks a fully semantic-based forwarding strategy and relies on geographical IDs, which aligns more with IP networks than NDN. This reduces SICN's completeness as a fully semantic system, especially in caching. To address the issues raised by [22], the state-of-the-art and previous work of the SANR-CMF, INF-NDN IoT [23] enhances NDN naming schemes by employing NLP techniques and selecting supernodes and ordinary nodes within the network. INF-NDN IoT assigns semantic tags, serving as forwarding clues, to both content names and supernodes, enabling semantic-based forwarding. However, INF-NDN IoT was unsuitable for a very large network and authors left the semantic cache

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

Content Names	EMP
smart/healthcare/system11/cameraABC/monitoring/data/city/hospital 1: smart/healthcare/system11/VideocameraABC/supervision/data/city/hospital 2: smart/health/system11/sameraABC/surveillance/data/city/hospital	✗
hongikuniversity/Sejong/bcn/profkim/publications 1: hongikuni/Sejong/bcnlab/drkim/publications	✗
sportsareaA/BuildingC/Camera3/recording 1: sportsareaA/BuildingC/Camera3/recording	✓
Sensor9021/reading/time08:00 1: Sensor9021/reading/time08:00	✓

Fig. 2. EMP description.

for future work. Another study [24] applies NLP to naming within an ICN-based weather monitoring system. In this approach, a monitoring camera captures video or images, and AI technologies generate appropriate content names based on the scenarios depicted. On the user side, the proposed scheme analyzes priority words from user-input sentences using NLP to retrieve content. The authors classify words into various labels with high probability. However, the study overlooks cache from a semantic perspective.

D. Summary of Related Work

Many scholarly papers have explored the cache schemes in IoT, tailored to specific scenarios. However, these improvements are limited by factors, such as name ambiguities and errors in the EMP mechanism of content names. These errors include issues such as phonetic similarities, spelling variations, titles, initials, and honorifics, etc. In addition, how to semantically select the specific content, where to perfectly place the selected content, and the replacement of content for cache performance have been an open challenge. EMP selects what content to retrieve for the cache without semantic awareness of content names. The placement strategy determines where content is cached by overlooking content centrality, while the replacement policy handles the eviction of content when it expires or when space is needed for new content. The NDN architecture relies heavily on content names, which are structured as English hierarchical sentences. This reliance means that accurately matching, selecting, placing, and replacing these content names necessitates a deep understanding of their semantics. Therefore, semantic awareness of content names is crucial for caching decisions in NDN. The inadequate combination of semantics and cache policies can significantly hinder the cache hits which is critical in IoT applications, such as military, healthcare, and transportation, where cache hit is crucial. To address the challenges of naming and caching policies, further research is vital to develop solutions. Therefore, SANR-CMF specifically addresses naming resolutions that are linked to cache decisions, an aspect that has been largely overlooked in previous literature. To the best of our knowledge, we are the first to propose a semantic-aware naming resolution specifically for caching purposes, along with a semantic-supported approach for content selection, placement, and replacement.

III. Motivation

Current in-network caching schemes lack semantic attributes, operating solely on an EMP for content names in their CS tables. There is a growing acknowledgment of the limitations inherent in this approach, which often leads to missed caching opportunities and inefficient data retrieval. Content names vary across different environments and applications, resulting in potential discrepancies that hinder cache performance under the strict confines of the EMP. For instance, content may exist in the cache and fail to be retrieved due to slight differences in naming conventions. This results in unnecessary use of network resources for further forwarding. This naming ambiguity leads to cache misses, denying the timely sharing of data with requesters. To highlight naming ambiguities, SANR-CMF focuses on a specific use case: retrieving data from the CS table where the EMP is enforced. Each row of the table contains multiple content names, as illustrated in Fig. 2. The first content name in each row represents the content name already stored in the CS table, while the remaining content names correspond to incoming content requests for exact matching. Incoming names are accompanied by numerical values, indicating the frequency of their requests to the CS table.

Fig. 2 illustrates that rows 3 and 4 conform to the EMP and are retrieved successfully (indicated by a green tick). On the contrary, rows 1 and 2, despite minor differences in their names, however, share identical semantic meanings and are unnecessarily forwarded to the FIB (highlighted with the red cross). The rationale is that the EMP failed to consider content names due to slight differences in content names. Moreover, in vanilla NDN, content selection favors rows 3 and 4 due to their single request instances, whereas row 1, despite being requested twice, remains unretrieved due to naming ambiguity, illustrating the pitfalls of the current approach in effectively managing content caching and retrieval. On the contrary, by applying the SANR-CMF to the described scenario, the data in row 1 will be identified as the MSRR.

Additionally, an IoT-driven device generates a request to the network at a specific instance. The proposed scheme calculates and assigns the optimal position on factors, such as content centrality. In contrast to legacy literature, the proposed scheme emphasizes a content-centrality approach over a topology-centrality approach for selecting cache nodes to improve the

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

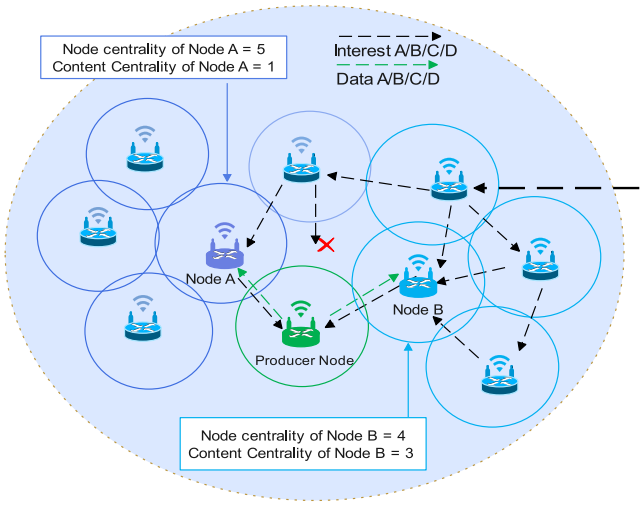


Fig. 3. Content centrality approach.

overall cache performance. The proposed scheme clarifies the claim by following the scenario, presented in the Fig. 3. Nodes A and B are two nodes in the IoT environment. The proposed scheme defines the content centrality for MSRR "A/B/C/D" of a node in the network as the count of interest packets for "A/B/C/D" received by a node from distinct vicinity nodes. Node A has five neighboring nodes in its vicinity and holds a strong central position, as illustrated in Fig. 3. On the contrary, node B has 4 nodes in its vicinity and a low degree of centrality. Meanwhile, node B receives three interest packets from distinct neighbors for MSRR. Node A receives just one interest packet for MSRR; if both nodes A and B receive data packets simultaneously, node B will be more suitable for caching based on content centrality. Here are the key points related to content centrality.

1) Content centrality for the content name is always lower or identical to the degree of centrality of the corresponding node.
2) A single node can possess a single degree of centrality at any moment. However, it can maintain multiple name centralities, with one assigned to each content name.
3) Node centrality is based on the topology of the network, while content centrality is determined for each specific content name.

The proposed scheme places the MSRR on the corresponding content-centric nodes, a strategy referred to as content-centric placement. These nodes, which hold the corresponding MSRR, are presented as CCN.

A. Why NLP for Caching?

Traditional EMP of NDN fails to understand semantic variations in content names (human-readable text data stored in CS table), such as abbreviations, synonyms, minor spelling differences, and other name ambiguities, leading to cache mismatching and excessive request forwarding. To this end, NLP is an AI-based computational approach for understanding text data, offering several solutions to perform semantic analysis. This makes NLP a natural fit for enhancing NDN's caching mechanism, allowing for more intelligent content matching for cache purposes. Leveraging the capabilities of NLP, SANR-CMF proposes an NLP-based approach to compute semantic similarity between NDN IoT content names, ensuring that semantically identical requests are matched and retrieved efficiently from the CS table. Additionally, the proposed NLP approach selects the MSRR as content to be cached for future requests on the locations where they will be in high demand as concept of content centrality rather than static network topology. By mitigating naming ambiguities and improving cache hit ratios (CHRs), NLP enhances the efficiency and adaptability of NDN IoT content retrieval.

IV. Proposed Methodology

SANR-CMF is deployed on an NDN-based IoT model. To this end, SANR-CMF incorporates four key mechanisms: content naming resolution, content selection, content placement, and finally content replacement, as demonstrated in Fig. 4. The foremost core functionality is the naming resolution for content matching, facilitated by advanced NLP techniques and represented by silver lake blue color (details outlined in Section IV-A). SANR-CMF proposes NLP-driven methods to match content names, specifically identifying those labeled as 0, 1, 2, and 3. Content names classified under Label 2 are selected for data retrieval and prioritized for content selection (shown by the silver lake blue color and briefly discussed in Section IV-B). For determining the nodes for content placement, SANR-CMF employs a GNN, using MSRR as input to identify content-centric nodes (presented by cerulean blue color and explained in Section IV-C). The final core functionality involves content replacement, executed on label 0 content names (depicted by vista blue color and detailed description in Section IV-D). A comprehensive breakdown of these mechanisms is presented in the following sections.

A. Semantic-Aware Naming Resolution

To measure the semantics similarity between the content names of the requested and already available content names in the cache storage, SANR-CMF introduces an effective strategy that integrates and modifies distinct NLP approaches. Initially, SANR-CMF utilizes bigram and trigram word-level semantics to estimate sentence-level (NDN content name in our case) semantic similarity. The rationale for utilizing bigram and trigram word-level semantics lies in the structure of the content names. Content names in IoT are often transient, fleeting, and diverse, emerging from various locations with distinct priorities. In essence, it becomes imperative to carefully examine the word-to-word relations within content names and extract valuable semantics from all conceivable aspects. To this end, the proposed scheme extracts information by organizing the content names into a set of synonymous words and hierarchical relationships, leveraging the hierarchical nature of the NDN namespace. Afterward, SANR-CMF takes into account the parts-of-speech (POS) tags embedded within content names by harnessing again the hierarchical nature. Finally, SANR-CMF extracts the semantic information from content names by embedding content names as dense vectors in continuous vector space, where words with similar meanings are closer together in the vector space.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

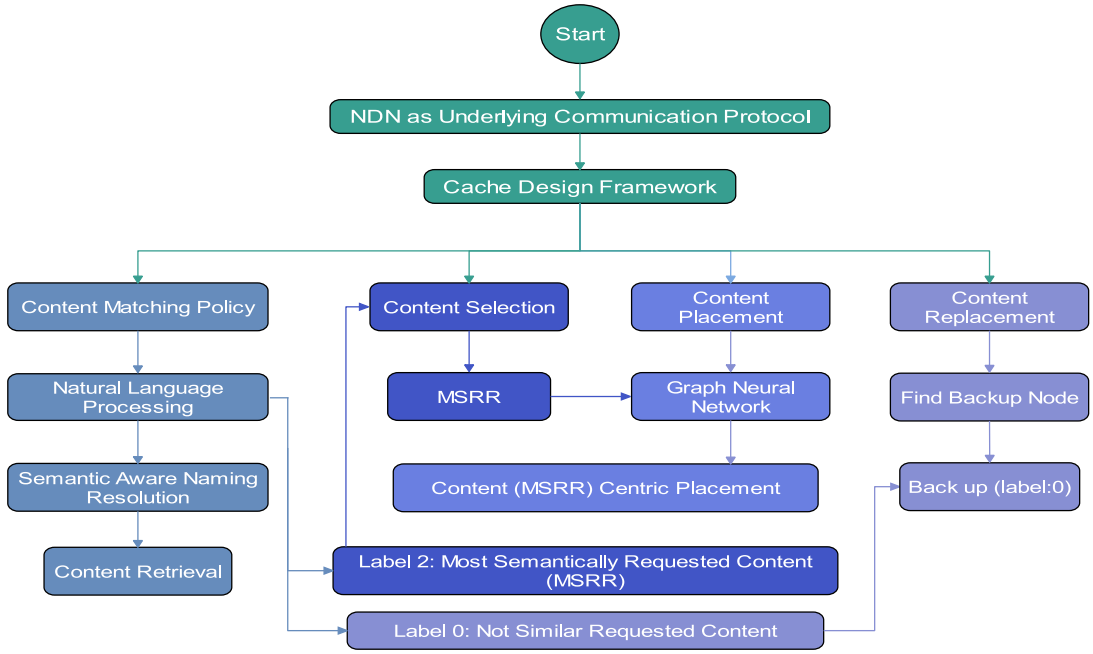


Fig. 4. SANR-CMF design framework.

To accomplish the aforementioned objectives, the semantic match policy process consists of three essential phases: preprocessing of content name, similarity analysis, and evaluation of similarity scores. In a given content name, preprocessing starts by removing punctuation marks from each content name. Following that, stop words (words that carry minimal semantic meaning, such as "a" and "the") are filtered out using indri's stop words collection [25]. Additionally, NLTK WordNet lemmatizer [26] is employed to transform words into their base forms. For instance, assume the NDN name: "/ndn/example/news/article/on/the/ongoing/development/of/artificial/intelligence /in/2024." The outcome of the preprocessing phase is a set of words: Set of words = "/ndn/example/news/article/ongo/develop/artifici /intelligen/2024." Common stop words, such as "on," "the," "of," "in" and "and" are removed. Additionally, some words are stemmed to their base forms (e.g., "development" to "develop," "intelligence" to "intelligen," "ongoing" to "ong"). To understand the process of similarity assessment between two content names, let's assume the name CS_{n1} already exists in the CS, and it will be matched with the name Int_{n1} for caching purposes. The nuances of each similarity measure are outlined in Table II. First, SANR-CMF exploits word's semantic to estimate the complete content name-level similarity. To this end, SANR-CMF utilizes WordNet [27] to compute complete content name semantic similarity. WordNet is a lexical database of the English language that organizes content name's words into sets of synonyms called synsets, each expressing a distinct concept. SANR-CMF proposes synonyms-based similarity SN_{SIM} defined as follows:

$$\text{SN}_{\text{SIM}}\left(\text{CS}_{n_1}, \text{Int}_{n_1}\right) = \frac{\sum_{w \in \text{CS}_{n_1}} \text{Max}\left(\underset{\text{v} \in \text{Int}_{n_1}}{\text{sim}_{\text{lch}}(\text{w}, \text{v})}\right) \cdot \text{weight}(\text{w})}{\sum_{w \in \text{CS}_{n_1}} \text{weight}(w)} \tag{1}$$

TABLE II
DESCRIPTION OF NOTIONS

Symbol	Description
N_words	Set of words obtained after processing NDN name string
AVS	N-dimensional mean feature vector of every NDN name
tc_s	Total count of words contained in the lexicon of `w2w_model` model for CS_{n_1} and Int_{n_1}
$w2w_model$	Learned word-embedding framework
$Vocab(w2w_model)$	Lexicon of `w2w_model` model
`add(t,` AVS`, w2w_model)`	Merging the N-dimensional vector associated with term t into AVS
`divide(`AVS `, tc_S)`	Distributing each value of AVS by tc_s
μ	The average of the components in the average vector AVS
σ	Standard deviation of the components in vector AVS

where $\text{Max}(\underset{\text{v} \in \text{Int}_{n_1}}{\text{sim}_{\text{lch}}(\text{w}, \text{v})})$ returns the maximum similarity score of word w in name CS_{n1} with any of the words v existing in name Int_{n1}. lch denotes the Leacock-Chodorow similarity mechanism [28]. Additionally, the weight assigned to each w in CS_{n1} is its inverse document frequency (IDF). IDF is defined as:

$$\text{IDF}(w) = \log\left(\frac{N_d}{1 + \text{DF}(w)}\right) \tag{2}$$

where N_d is the total number of content names in the dataset and $\text{DF}(w)$ is the number of content names in which the term w appears. The similarity function $\text{sim}_{\text{lch}(w,v)}$ is Leacock-Chodorow similarity defined as follows:

$$\text{sim}_{\text{lch}}(w, v) = -\log\left(\frac{d(w, v)}{2 \cdot D}\right) \tag{3}$$

where $d(w, v)$ is the shortest distance from concept w to concept v. The maximum depth is presented by $2.D$ in (3).

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

SANR-CMF leverages the hierarchical nature of content names and estimates the similarity by considering POS tags present in the hierarchical sequence of NDN names. POS tagging assigns syntactic categories (e.g., nouns, verbs, adjectives) to words in content names based on their grammatical roles. Words within content names that have identical POS tags are grouped into the same class. Therefore, similarity can be determined when two words, each from separate content names, share identical POS tags. Such as, `/home/living$room/lights` and `/home/living_room/appliance` are two different names within the same class based on their shared characteristics or POS tags. The average hierarchical pairwise-based similarity HP_{SIM} is defined as follows:

$$\mathrm{HP_{SIM}}\left(\mathrm{CS}_{n_1}, \mathrm{Int}_{n_1}\right) = \frac{\sum_{w\in \mathrm{CS_{n1}}} \mathrm{Max}\left(\mathrm{sim}_{v\in \mathrm{Int_{n1}}}\left(w, \mathrm{tag}_w, v, wv_{\mathrm{model}}\right)\right)}{|N|}. \quad (4)$$

The complete function $\mathrm{sim}_{v\in \mathrm{Int_{n1}}}(w, \mathrm{tag}_w, v, wv_{\mathrm{model}})$ in equation (4) represents the similarity based on the word-embedding model wv_{model} between two words, w and v, which have identical POS tags. Here, the tag_w signifies the POS of word w, which must match the POS of the word v. At the last, SANR-CMF initially generates the feature vector for each word and subsequently constructs a feature vector for the content name N by averaging the word vectors. The average feature vector *AVS* per content name is then computed. For obtained *N_words*, SANR-CMF computes *AVS* for each content name. The word-embedding model generates an N-dimensional vector for each word in the content name. Consequently, SANR-CMF acquires the feature vector for each word that belongs to *N_words*. The vectors are computed solely for words within the vocabulary of *Vocab*(*w2w_model*) [29]. Subsequently, the feature vectors for each word within a specific content name are aggregated. The resulting vector after aggregation is stored in *AVS* for the respective content name. Each value in *AVS* is subsequently divided by the total number of words in the corresponding name. Although averaging the vectors may obscure certain distinctions, some may be significant while others may not. Therefore, each element of the *AVS* vector is normalized on a specific scale. This normalization is performed utilizing the z-score normalization technique

$$x' = \frac{x-\mu}{\sigma}. \quad (5)$$

In this process, the normalized value and the original value are denoted by x' and x, respectively. Here, σ represents the mean, and μ denotes the standard deviation of the vector's elements. Following normalization, SANR-CMF utilizes the cosine similarity measure to calculate the similarity score between CS_{n1} and Int_{n1}.

Let *Nor_AV1* and *Nor_AV2* represent the two normalized average feature vectors calculated for the respective content names CS_{n1} and $\mathrm{Int_{n1}}$ intended for similarity measurement. The similarity measure based on average feature vectors $Avg_FV_S_{\mathrm{SIM}}$ is defined as follows:

$$\mathrm{Avg_FV_S_{SIM}} = \frac{Nor_AV1 \cdot Nor_AV2}{\|Nor_AV1\| \cdot \|Nor_AV2\|} \quad (6)$$

$$= \frac{\sum_{i=0}^{N} Nor_AV1_i \cdot Nor_AV2_i}{\sqrt{\sum_{i=0}^{N} Nor_AV1_i^2} \cdot \sqrt{\sum_{i=0}^{N} Nor_AV2_i^2}} \quad (7)$$

where Nor_AV1_i and Nor_AV2_i denote the ith feature value of the vectors *Nor_AV1* and *Nor_AV2*, respectively. The dimension of the vector *AVS* is denoted by N.

For the final semantically score estimation, each unique metric contributes variably to determining the similarity between content names. Hence, we employ a supervised feature selection technique called ElasticNet [30] to gauge the significance of each measure. We consider the various similarity measures as a feature set. This process is divided into two phases: 1) importance estimation and 2) linear ranking. ElasticNet, a strategy in regularized regression, merges the l_1 and l_2 regularization penalties of the Lasso and Ridge methods [31], respectively. It can be expressed as a combination of Lasso and Ridge regression, defined as follows:

$$\min_{\beta\beta_0\in\mathbb{R}^{p+1}} \left[\frac{1}{2}\sum_{i=1}^{n}(y_i - \beta_0 - x_i\beta)^2 + \lambda P_\alpha(\beta)\right] = \sum_{j=1}^{p}\left[\frac{1}{2}\lambda(1-\alpha)\beta_j^2 + \lambda\alpha\left|\beta_j\right|\right] \quad (8)$$

where

$$P_\alpha(\beta) = (1-\alpha)\frac{1}{2}\|\beta\|_2^2 + \alpha\|\beta\|l_1. \quad (9)$$

y_i represents the response of observations i, while x_i denotes the data. p and β represent the dimension of the feature space and coefficients associated with each feature in the model, respectively. The parameters of the linear regression are denoted by α. When α equals 0 or 1, the ElasticNet simplifies to Ridge regression or Lasso, respectively. Ridge regression maintains all explanatory variables in the model due to the smoothness of the l_2 norm. Conversely, Lasso offers a concise representation of the feature space due to the sparsity induced by the l_1 constraints. In the final step, a linear ranking approach incorporates all similarity measures along with their respective weights to calculate the final similarity score between Int_{n1} and CS_{n1}

$$\mathrm{sim}\left(\mathrm{CS}_{n_1}, \mathrm{Int}_{n_1}\right) = \frac{\sum_{i=1}^{T} w_i \cdot SM_i\left(\mathrm{CS}_{n_1}, \mathrm{Int}_{n_1}\right)}{\sum_{i=1}^{T} w_i} \quad (10)$$

where $\mathrm{sim}(\mathrm{CS}_{n1}, \mathrm{Int}_{n1})$ denotes the ith similarity measure, and w_i represents the importance score of the ith corresponding measure estimated in (10). As a result, the final output is a continuous semantic similarity score ranging from 0 to 1. To interpret these scores meaningfully, we discretize the continuous score into four semantic labels. The threshold values were chosen based on practical observations of score distributions and manual examination of representative NDN content name pairs, ensuring that the assigned labels align with

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

perceived semantic similarity. The four semantic labels with score ranging criteria are defined as follows.

1) *Label 0:* Semantically not similar (score range: 0.0–0.25).
2) *Label 1:* Semantically roughly similar (score range: 0.26–0.5).
3) *Label 2:* Semantically completely similar (score range: 0.8–1.0).
4) *Label 3:* Semantically similar (score range: 0.51–0.79).

It's essential to note that SANR-CMF prefers to retrieve data from CS solely when the output label is 2. This decision is based on the observation that labels 1 and 3 may result in incomplete or less semantic matches between content names. Such mismatches could lead to lower interest packet satisfaction rates and trigger retransmissions upon consumer request. Conversely, there are scenarios where labels 1 and 3 might successfully meet consumer requirements. Consequently, data retrieval using labels 1 and 3 is designated as application-specific. Applications can leverage these labels in cases where the likelihood of retransmissions is minimal, optimizing their usage based on specific contextual needs. To mitigate retransmission, SANR-CMF prioritizes label 2. Using label 2 ensures that the consumer receives precise data (can be with different names) rather than similar alternatives. Furthermore, SANR-CMF may forwards the request to FIB in the case of label 0.

B. Semantic Content Selection

This is the second mechanism of SANR-CMF. In contrast to other methods, SANR-CMF prefers semantically selection of content for cache. Semantical content selection refers to the selection of semantically repeated request content. Content that is repeatedly requested since it aligns with consumers' semantic preferences is prioritized for selection. Such prioritized contents are MSRR. MSRR is chosen based on its recent access history integrated with label 2, specifically the number of times it has been retrieved after NLP operation. In SANR-CMF, each node autonomously keeps a discrete sizing table that logs incoming requests. This table is used to identify frequently accessed content by recording the MSRR names and their respective request frequencies. For each content, the network node must track the number of incoming requests. Content that has an NLP-based label 2 (Section IV-A) with a higher frequency of downloads (number of recent requests) is identified as semantically frequent requests. Frequently requested content is identified by calculating the total count of incoming requests for specific content on a node. The PIT entry record serves to count the total requests made for each content name. When the request count for a particular content name exceeds that of all others, it is classified as the MSRR. In contrast to other schemes, SANR-CMF selects a group of MSRRs to boost data availability. These MSRRs are recommended for caching at CCN, as described in the next section. The SANR-CMF selects the MSRR by counting how often each MSRR is requested. It lists all accessed content from least to most accessed. Afterward, SANR-CMF chooses the top one-fifth of the most accessed MSRR to store at CCN in the network, making the content faster to retrieve. Assume CS has 10 contents in the list. The top 2 will be selected as MSRR.

Algorithm 1 CCN Selection

Input: Network graph $G(V, E)$, Feature vectors F, Time intervals $T = 8$, GNN layers $K = 2$
Output: Selected cache nodes *CCN*
Procedure: Temporal and Spatial Analysis
Step 1: Temporal Analysis using 1D-CNN
for each node $v \in V$ **do**
    Construct feature vector $F_v = [f_v^{t-7}, ..., f_v^t]$ ▷ Last 8 time intervals
    Pass F_v through two convolutional layers:
    - Conv1 (Kernel size = 2) + ReLU
    - Conv2 (Kernel size = 6) + ReLU
    Apply fully connected layer with ReLU activation
    Output: Anticipated MSRR count $\hat{f}_v^{t+1}$ for the next interval
end for
Initialize node features $\mathbf{h}_v^{(0)} = \hat{f}_v^{t+1}$
Step 2: Spatial Aggregation using GNN (GraphSAGE)
for $k = 1$ to K **do** ▷ 2 message-passing layers
    for each node $v \in V$ **do**
        Aggregate features from neighbors up to k-hops:
    end for
end for
Step 3: Cache node Selection
Rank nodes based on $\mathbf{h}_v^{(K)}$ and $\hat{f}_v^{t+1}$
Select top nodes with the highest anticipated MSRR counts as *CCN*
return: *CCN*
End: *CCN* Selection
Terminate

C. Content-Centric Placement

The content that is most frequently requested based on semantic relevance is transmitted to the end user from which the request originated. To optimize this process, it is crucial to identify strategic caching positions along the publisher-subscriber path. Consequently, factors, such as path distance, path or route elongation factor (REF), and optimal caching locations must be considered to effectively place the most frequently requested content.

To find the CCN in the network with the corresponding MSRR, the proposed scheme leverages the deep neural network (DNN). CCN selection mechanism is highlighted in Algorithm 1. The implemented DNN-based framework consists of 1D-CNN layers alongside GraphSAGE layers. The 1D-CNN is specifically designed to capture the temporal relationships of semantically recurring request patterns at each node. For every content name, the input to the 1D-CNN is represented by a feature vector that encapsulates data for each node. This vector comprises the count of requests for the MSRR from that specific node across the

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2 Placement of MSRR at CCN

```
Input: MSRR - most semantically repeated requests, Path from requester to provider
Output: MSRR successfully placed at CCN
Procedure: (MSRR, Path)
MSRR lifetime (C_LT) = 0 seconds,
MSRR maximum lifetime (C_MLT) = 4 seconds
Path = route from requester to provider
for a node in path do
    if node = CCN then
        node content + = MSRR
        node-MSRR (C_LT) = C_LT
        if C_LT < C_MLT then
            C_LT+ = 1
        end if
    end if
end for
return: MSRR placed at CCN
Terminate
```

last eight time intervals. The output generated by the 1D-CNN indicates the anticipated number of requests for MSRR from that node in the next time interval. The structure of the 1D-CNN includes two convolutional layers with kernel sizes of 2 and 6, respectively. Each of these convolutional layers is followed by a rectified linear unit (ReLU) activation layer. After the second convolutional layer, a fully connected layer employs a ReLU activation function. The main aim of the 1D-CNN is to predict the number of requests for MSRR per node in the upcoming time slot. Building upon the 1D-CNN architecture, the proposed methodology utilizes a graph neural network (GNN) to capture the spatial relationships among node features. A GNN accomplishes this by enabling message passing between nodes. Within a network topology, each node aggregates features from its neighboring nodes, viewing the network as a directed graph with nodes referencing their adjacent nodes. The GNN model uses two message-passing mechanisms, indicating that the CCN can aggregate (pooling) information from nodes up to two hops away, as expressed in the following equation:

$$\mathbf{h}_v^{(k)} = \sigma\left(\mathbf{W}^{(k)} \cdot \mathrm{AGGR}\left(\{\mathbf{h}_u^{(k-1)} \mid u \in \mathcal{N}(v) \cup \{v\}\}\right)\right). \quad (11)$$

Equation (11) describes how node v's feature vector is updated at the kth layer in a GNN. $\mathbf{h}_v^{(k)}$ is the updated feature vector.

The activation function σ is applied after aggregating the features. $\mathbf{W}^{(k)}$ is the learnable weight matrix at the kth layer, and $\mathrm{AGGR}(\{\mathbf{h}_u^{(k-1)}{:}u \in \mathcal{N}(v) \cup \{v\}\})$ represents the aggregation of features from node v's neighbors and itself from the previous layer. First message passing, the target node collects features from its 1-hop neighbor nodes. Once aggregation is complete, every node's feature vector is adjusted. In the second message-passing layer, the target node performs aggregation on information from its 2-hop neighboring nodes, as its 1-hop neighbors have already gathered features from their respective adjacent nodes during the first layer. Following the last message-passing layer, the node features are adjusted and then employed to predict the labels of the nodes. Regardless of the network's configuration, the GNN's primary objective is to identify CCNs and aggregate the features of neighboring nodes for each CCN to facilitate caching. The cached content across these CCNs is linked with a timestamp. Content is cached for a specified duration to enhance caching capacity, with the duration increasing as the request traverses toward the end devices. Simply, caching time is longer for nodes closer to consumers due to frequently requested content being chosen as MSRR again, resetting the timestamp. On the contrary, content far from the requester has a shorter lifespan, as subsequent requests will be fulfilled by the closest node. Furthermore, to optimize data availability, a duplicate of the requested data is stored on the edge nodes. This allows following requests by end users to be instantly fulfilled by these edge nodes.

To mitigate content redundancy and elongation factor, each content name is assigned a specific expiry time, with a maximum expiry time of 4 s. This limitation ensures that data is unable to be stored for longer than 4 s at any node. Caching serves to temporarily store content, with cache capacity typically measured by the duration for which content is cached. Therefore, specifying the content's lifespan is essential in NDN IoT networks to minimize redundancy and enhance the hit rate. One potential work [32] suggests that caching content for 10 s can save up to 60% of network bandwidth by preventing redundant content transmission. However, in the SANR-CMF system, content can be stored at various nodes along the delivery trajectory. To avoid pointless cache utilization, the caching duration is set to 4 s. Longer caching durations may hinder the accommodation of new MSRR and increase the number of requests forwarded to distant content providers, leading to path congestion. Conversely, shorter caching durations result in more cache evictions and redundancy. The optimal caching duration depends on traffic characteristics, such as temporal locality, ensuring efficient cache utilization for semantically demanded content.

Algorithm 2 outlines the content caching mechanism of SANR-CMF, where CCN are designated to cache the MSRR. A CCN is a central point that may have requested to download MSRR. The entire route may include multiple CCNs. However, to minimize content redundancy and the elongation factor, a content lifetime (C_LT) is associated with every content, with the highest lifetime C_MLT at 4 s. MSRR can be stored at various nodes along the data flow pathway to optimize cache utilization. Higher value parameters for C_LT lead to content remaining in the cache for a higher time, reducing the possibility of accommodating new MSRR and requiring many subsequent requests to be forwarded to distant content providers, thereby increasing the REF. Conversely, smaller C_LT values lead to more ejections and higher redundancy. Furthermore, C_LT is influenced by the traffic dynamics within the domain, particularly the temporal proximity of the traffic patterns [33]. Consequently, the cache operates efficiently by facilitating the MSRR.

Fig. 5 illustrates the content replacement mechanism within CCN, where CCN manages the backup caching process at neighboring nodes. Within SANR-CMF, CCN possesses the additional capability to store content on neighboring nodes.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

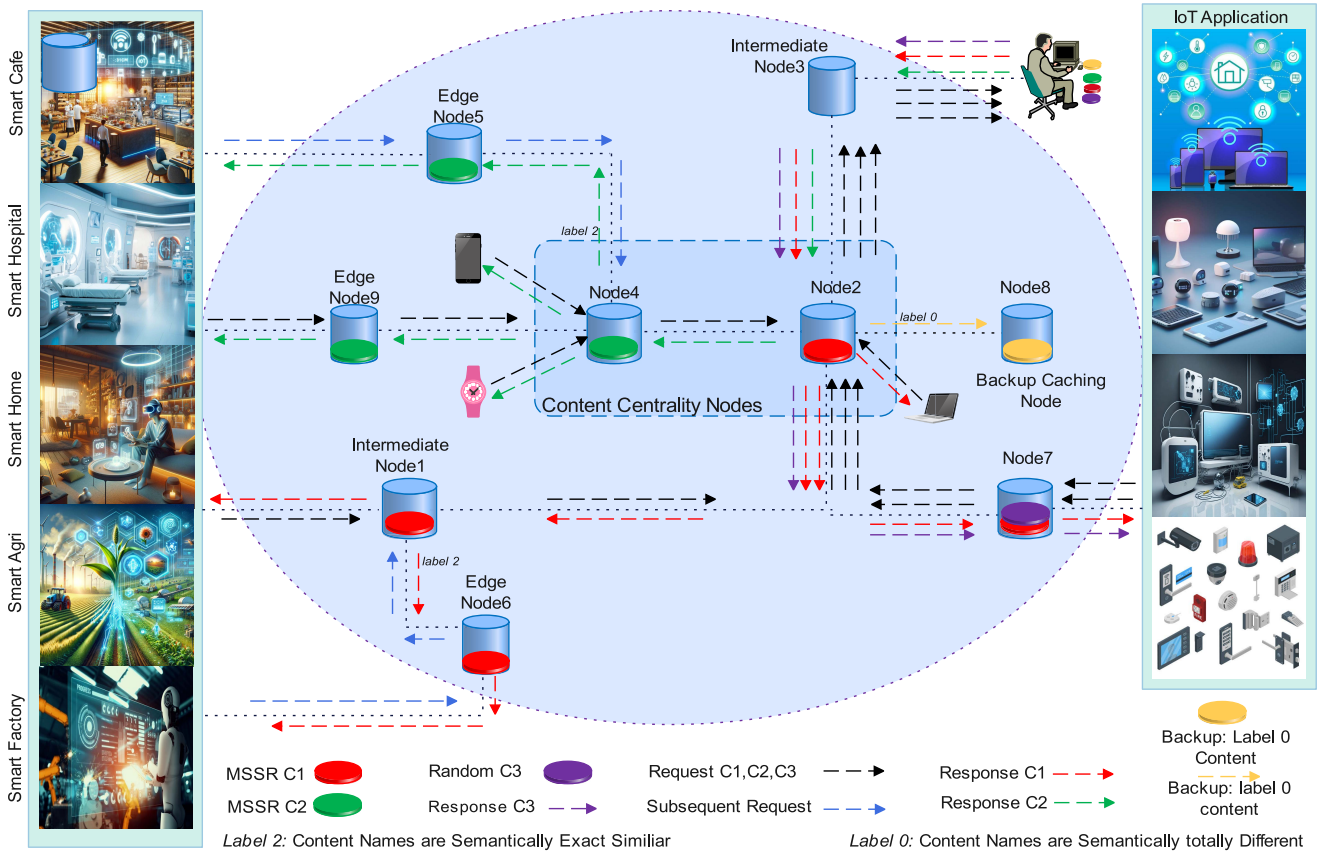


Fig. 5. SANR-CMF caching strategy: a case study scenario.

This mechanism is executed when new content arrives at a CCN node with a full CS capacity. CCN ejects the content and examines neighboring nodes for available cache storage. The ejected content is subsequently stored at the neighbor node with the most available cache capacity. The content selected for eviction is semantically totally different from the newly arrived content, which is then cached at a backup node. This strategy has no effect on content retrieval latency or CHR, as SANR-CMF evicts content based on the least semantically accessed criterion, thus facilitating the caching of new content at the CCN. As a result, the semantic popularity of the ejected content remains consistent at CCN and backup caching nodes as well, thereby optimizing caching effectiveness.

D. Content Replacement

Content replacement is initiated at the moment when the cache storing capacity of CCN reaches its full capacity.

Fig. 5 presents the content placement alongside the approach of efficient content replacement. In the illustration, a request for C1 is directed to node 1 and node 7, and a request for C2 is directed to node 9. Meanwhile, two more requests for C2 are sent to node 4. A network has recently been initiated. Therefore, the CS table is empty and has no data to calculate the semantic similarity between content names. As a result, CS forwards requests to the content producer to retrieve content C1 and C2. Swiftly, the content producer acknowledges the request by transmitting content C1 and C2 to the requester. First-time requests for any content (such as C1 and C2) are represented by black color dotted arrows, while the response for content C1 and C2 are indicated by red and green color dotted arrows, respectively. Upon receipt of the request for C1 and C2, the producer promptly dispatches content C2 via nodes 3, 2, and 4. Meanwhile, content C1 is dispatched through two possible paths: nodes 3, 2, 1 or nodes 3, 2, and 7. All corresponding nodes receive the data and forward it to the consumers without checking the semantics of content names.

Afterward, the network enters the second phase, where CS tables of the aforementioned nodes are full of content names with corresponding semantic assessments. At this stage, after NLP and DNN procedures, we assume both contents C1 and C2 are identified as MSRR and their corresponding content centrality nodes are nodes 2 and 4, respectively. As per the content caching framework of the proposed methodology, C1 and C2 are cached at content centrality nodes 2 and 4. Additionally, copies of both C1 and C2 are available at intermediate node 1 (node 1 serves as an intermediate node for disseminating C1 and C2) and node 7.

To accommodate subsequent data dissemination, requests are generated and routed toward node 6 and node 5 to retrieve C1 and C2, as represented by blue dotted arrows in Fig. 5. For the sake of understanding the concept, we posit that the possibility exists of naming ambiguities in the content names of subsequent requests, such as C1* and C2*. Nodes

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

6 and node 5 further forward the requests to neighbor node 1 and node 4, respectively. CS of node 1 and node 4 hold the corresponding data with slight variation in the names. Therefore, instead of forwarding C1* and C2* to FIB, node 1 and node 4 first apply the proposed naming resolution on C1* and C2* from node 6 and node 5, respectively. After applying the proposed NLP operations, the result of matching content names lies in the category of label 2 (semantically completely similar). Therefore, data C1 and C2 are cached from the CS store of node 1 and node 4. Furthermore, both requests are fulfilled from the nearest cached copies of C1 and C2 (retrieved from node 4 and node 1, respectively). Simultaneously, node 7 receives a request to fetch content C3. While C3 is being transmitted from the content provider to the IoT device, an attempt is made to cache a copy at CCN 2. However, the cache at CCN 2 is already at storage capacity. Consequently, the content replacement mechanism is triggered to remove that content which is semantically different from C3. From a technical perspective, the proposed scheme will perform NLP and choose any content whose semantic result is labeled 0, assuming C4 in our case. There are chances that C4 can be MSRR for any other request. Therefore, instead of removing C4, it is cached as the backup node 8. Backup caching is presented with a yellow arrow. Node 8 is chosen as the backup caching node due to its larger free-cache space. Content C4 is then cached at node 8 after being replaced from CCN2. Furthermore, in cases where the cache of the CCN reaches high capacity, the content will be stored at the neighboring node, ensuring the fulfillment of the following requests. This approach effectively reduces the distance range between the end user and the content publisher, leading to an increase in cache hits. As a result, the overall caching performance is enhanced, contributing to improved efficiency.

E. Complexity Analysis

The complexity of the proposed Algorithms 1 and 2 is analyzed step by step and combined to derive the overall complexity. Since both algorithms involve multiple key steps, we represent the complexity of each step separately and then compute the total complexity. The Algorithm 1 consists of three main steps: 1) Temporal Analysis; 2) Spatial Aggregation; and 3) Cache Node Selection. In the first step, each node constructs a feature vector over eight time intervals and processes it through a 1D-CNN with two convolutional layers and a fully connected layer. Given that each convolutional operation runs in $O(f)$, where f is the number of features, the total complexity for all nodes is $O(nf)$. In the second step, each node aggregates information from its neighbors up to two hops using the GraphSAGE model. The message passing operation involves aggregating from d neighbors in each layer, leading to a complexity of $O(nd^2f)$, where d is the average node degree. The final step ranks nodes based on their predicted MSRR values, which requires sorting with a worst-case complexity of $O(n \log n)$. Combining all steps together, the overall complexity of the CCN Selection algorithm is $O(nd^2f + n \log n)$. The GNN processing step dominates the complexity, making it efficient for sparse graphs.

The Algorithm 2 executes only when the cache reaches full capacity. It iterates over the request path, checking each node. In the worst case, the path length is H, leading to a complexity of $O(H)$, where H is the number of hops in the request path. Since H is at most $O(n)$, the worst-case complexity remains $O(n)$. Given that there are no additional computationally intensive steps, the final complexity of the Algorithm 2 is $O(n)$.

V. Performance Evaluation and Results

This section provides the performance evaluation and the results. The natural language tool kit (NLTK) [34] and Icarus simulator [35] are selected as the evaluation tools to assess the effectiveness of the proposed SANR-CMF strategy. Icarus is designed to evaluate caching and routing strategies in ICN and NDN-based IoT environments. In this simulator, operations are event-driven, with each event triggering a specific task. To handle requests from end devices, Dijkstra's shortest-path algorithm was employed in Icarus to find the least weighted route. Consequently, if the required content is available at CCN or a neighbor node of the CCN network node along the path, the request is fulfilled from the corresponding node, reducing latency. Icarus follows a model-view-controller (MVC) design pattern, which breaks the simulation into four essential modules: beginning with the scenario generation block, followed by the orchestration block, the execution block, and ending with the result collection block. Incoming requests are treated as events, and for each event, the timestamp, source, and destination are computed during execution. Simulation results are collected in the collection block, and overall latency is determined by summing the delays across all network links involved in content dissemination.

A. Simulation Setup

To train the semantic-aware naming resolution model, the stanford natural language inference (SNLI) [36] dataset is utilized. For further cache decisions, parameters and corresponding values are listed in Table III. To this end, the GEANT topology is selected featuring 35 nodes. GEANT is recognized as a real-world globally distributed academic network. The Zipf with alpha values of 0.55, 0.75, and 0.95, is preferred to represent low, average, and high semantic content popularity models, respectively. Initially, content is allocated and cached on the source nodes, while nodes with a degree of one are designated as end devices. To warm up the caches, 55 000 requests are generated, followed by another 55 000 requests to assess the performance of the system. In Icarus, every event represents a cycle where a request is directed and the corresponding content is delivered. The results are derived by ranging content numbers from 21 000 to 55 000. 0.45% is set as the cache size of the content numbers. Consequently, cache sizes vary between 90 and 248. The performance of SANR-CMF is compared with CaDaCa [16] and PoolCache [17] (discussed briefly in Section II-B). The rationales for choosing CaDaCa and PoolCache as baseline papers are their partial consideration of semantics in cache

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

TABLE III
SIMULATION PARAMETERS

Parameters	Value
Topology	GEANT
Nodes	35
Warm up requests	55000
Zipf values range	0.55, 0.75, 0.95
Ranging content numbers	21000 to 55000
Cache size set	0.45%
Cache size variation	90-248
Semantic label	2
Evaluation metrics	Semantic Satisfaction Rate, Cache Hit Ratio, Route Elongation Factor, Latency, Ablation Studies, and NLP Classification Performance

and strong citation scores, respectively. In our simulations, we consider the following performance metrics.

1) *SSR:* It refers to a metric that measures the extent to which users' content requests are met based on the semantic relevance of the retrieved content. It is calculated as the average of the relevance scores between the requested content and the retrieved content across all requests. The SSR is computed as follows:

$$\text{Semantic satisfaction rate} = \frac{\sum_{i=1}^{N} \mathcal{R}(C_i, R_i)}{N} \quad (12)$$

where N represents the total number of content requests, C_i is the ith content requested by the user, and R_i is the ith content retrieved from the cache. The relevance function $\mathcal{R}(C_i, R_i)$ measures the semantic similarity between the requested content C_i and the retrieved content R_i, quantifying how well the cached content aligns with the user's intent.

2) *CHR:* This metric quantifies the proportion of requests served from in-network caches and is calculated as stated below

$$\text{Cache hit ratio} = \frac{R_C}{R_C + R_O} \quad (13)$$

where R_C represents the counts of requests satisfied by the node's cache, and R_O refers to the number of requests retrieved from the actual publisher (original source). A higher CHR indicates a larger percentage of requests are dispensed from the cache, demonstrating improved caching efficiency.

3) *REF:* It measures how much longer a route taken by a request to a provider is compared to the optimal route leading to the serving cache node (CCN or edge nodes in our case). The formula used for computation is presented below

$$\text{Route elongation factor} = \frac{1}{N} \sum_{i=1}^{N} \frac{\mathcal{D}_{\text{actual},i}}{\mathcal{D}_{\text{optimal},i}} \quad (14)$$

where $\mathcal{R}_{\text{avg}}$ specifies the average elongation factor across all evaluated routes, N denotes the total number of requests generated, $\mathcal{D}_{\text{actual},i}$ represents the actual route length traversed by data for request i to actual producer, and $\mathcal{D}_{\text{optimal},i}$ indicates the shortest possible distance for request i from the source node to the serving caching node.

4) *Average Latency (Lat):* It alludes to the time, measured in milliseconds, consumed across various links to address incoming requests, measured by given formula

$$\text{Average latency} = \frac{1}{C} \sum_{j=1}^{C} t_j \quad (15)$$

where t_j denotes the duration taken by the jth request to locate the corresponding data, and C signifies the count of requests directed to the network.

5) *Ablation Studies:* To demonstrate the contributions and effectiveness of each component of SANR-CMF, ablation studies are performed by evaluating the CHR under the following three scenarios.
   a) Without semantic name matching.
   b) Without the DNN-based selection and placement mechanism.
   c) Without the proposed backup strategy.

6) *NLP Classification Performance (NCP):* To evaluate and compare NCP, three key metrics are employed in SANR-CMF, as follows.
   a) *Inference Time:* Refers to the time required by the model to predict once the input data is provided.
   b) *Model Size:* Refers to the amount of memory or storage required by the model.
   c) *Accuracy:* Measures the overall correctness of the model's predictions, as defined by

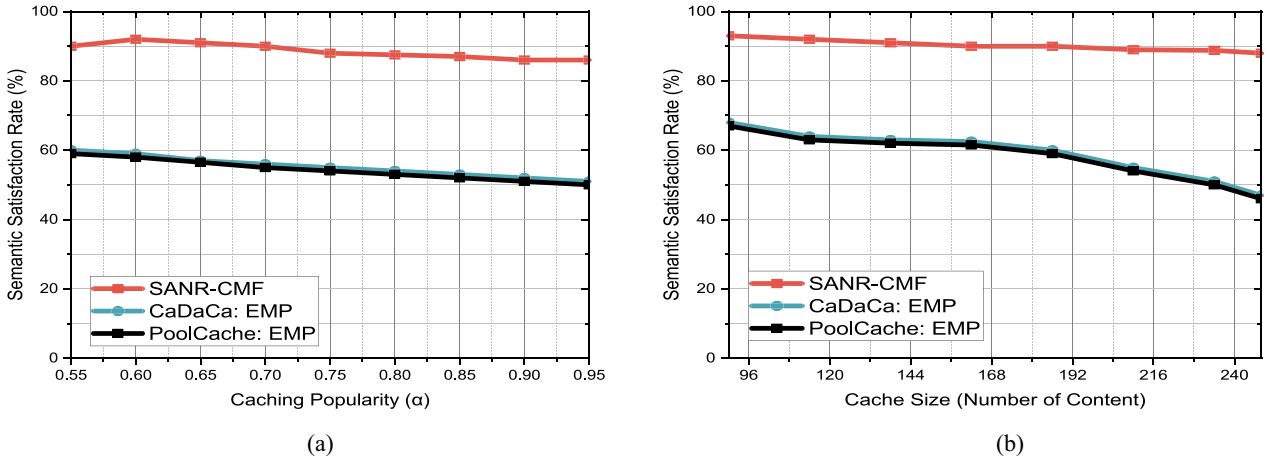
$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (16)$$

where TP represents true positives, TN describes true negatives, FP denotes false positives, and FN corresponds to false negatives.

B. Evaluation Results

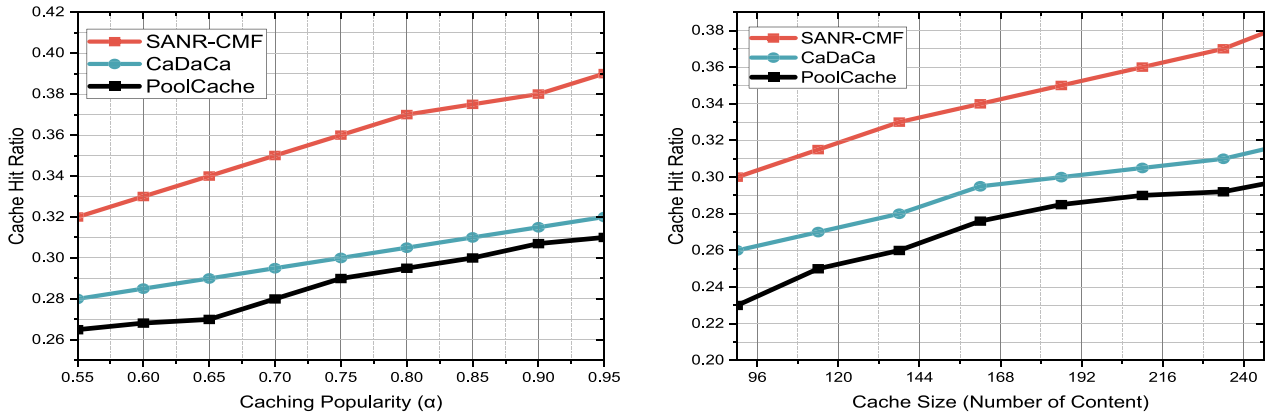
1) Semantic Satisfaction Rate: Fig. 6(a) and (b) illustrate the SSR on the y-axis as a function of cache sizes and alpha range on the x-axis. The results show that the SSR increases compared to CaDaCa and PoolCache, which adhere to the EMP. PoolCache and CaDaCa's EMP rely on string-by-string matching of content names; when a content name fails to precisely match, the request is forwarded to the FIB, overlooking the possibility that the content might already exist in the CS under slight variations in the name. This rigid approach diminishes SSR, as the system fails to exploit semantic similarities between content names. As a result, PoolCache and CaDaCa show similar performance and the SSR decreases significantly. On the contrary, SANR-CMF outperforms CaDaCa and PoolCache by fully leveraging advanced NLP techniques for intelligent naming resolution. SANR-CMF integrates semantic analysis from multiple perspectives, including POS, synonym recognition, word relations, and the hierarchical structure inherent in content names. This allows SANR-CMF to resolve content requests more flexibly and intelligently, matching user requests even when exact name matching fails. By utilizing semantic similarity measures, SANR-CMF can identify content that may be stored

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 6. SSR as a function of (a) caching popularity and (b) cache size.



(a) (b)

Fig. 7. CHRs as a function of (a) caching popularity and (b) cache size.

under slightly different names and shares the same meaning (under label 2, as discussed in Section IV-A), allowing SANR-CMF to achieve superior SSR performance, especially in dynamic and semantically rich environments. In this way, the data that EMP failed to retrieve can be successfully retrieved by SANR-CMF. A notable question arises regarding the SSR performance of PoolCache, CaDaCa's EMP, and SANR-CMF: if EMP completely overlooks semantics, why does its SSR fail to reach zero? Similarly, why does SANR-CMF's SSR remain below 100% despite fully incorporating semantics? The explanation lies in the fact that the SSR of EMP is unable to be zero, as any content that satisfies the exact matching criteria inherently fulfills semantic satisfaction. However, SSR decreases when content names may have ambiguities. Conversely, SANR-CMF's SSR fails to reach 100%. This is owing to the retrieval of content labeled as 2 in the naming resolution process, ensuring an exact match to the user's request. Labels 1 and 3 are excluded to prevent the retrieval of content that may not precisely align with the user's request and lead to retransmission in the network.

2) Cache Hit Ratio: Fig. 7(a) and (b) depict the relationship between the CHR and cache sizes, and alpha values, respectively. Each subfigure shows the impact of different alpha values ranging from 0.4 to 0.95 and cache sizes varying from 90 to 248. By increasing the cache size and alpha values the performance of all strategies improves. The rationale stems from the fact that a larger cache allows more content to be stored within the nodes, leading to a higher cache hit rate for both methodologies. Another reason for this improved efficiency is the high node-to-edge ratio in the GEANT topology. Therefore, there is a reduced likelihood of finding various routes to approach a node when utilizing arbitrarily chosen nodes from various data publishers. Overall, SANR-CMF outperforms CaDaCa and PoolCache by selecting a group of the MSRR to be stored along the data route, which enhances the accessibility of repeatedly requested content recently. Moreover, the semantic naming resolution used in SANR-CMF plays a crucial role in achieving cache hits. The rationale behind this lies in the fact that some content names overlooked by the EMP are effectively identified through the semantic naming resolution.

Additionally, SANR-CMF caches MSRR at the edge and CCN nodes, providing additional opportunities to respond to requests. By caching the MSRR at the appropriate CCN, further requests are more likely to be fulfilled by nearby or local CCNs. This reduces the need to forward requests to other nodes or the origin server, leading to more cache hits. On the contrary, CaDaCa caches data based on local node-based categories. If the data belongs to an existing category with a high probability and history, CaDaCa will cache it for future

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

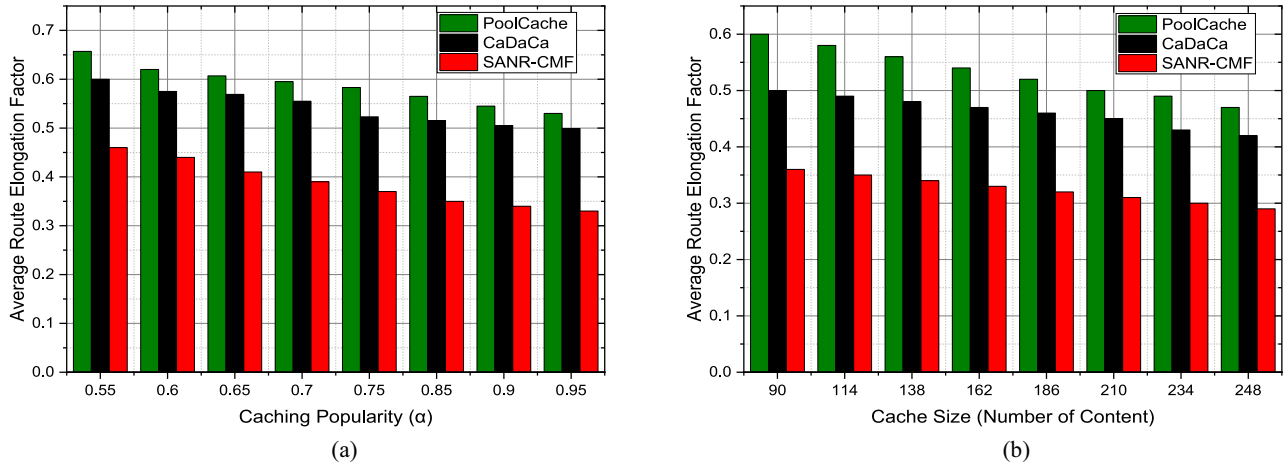


Fig. 8. REF as a function of (a) caching popularity and (b) cache size.

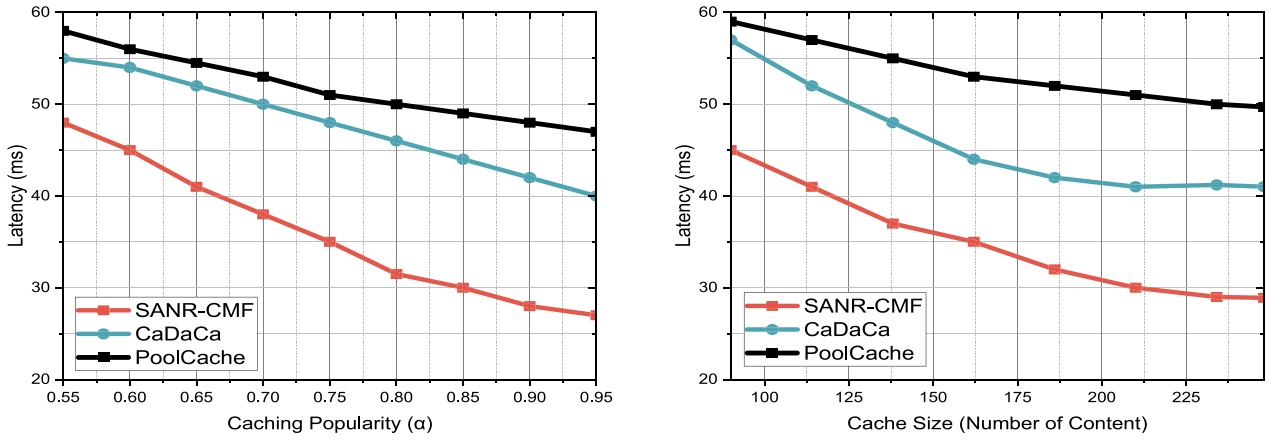
requests. If neither the data nor its category is available in the cache, requests must traverse multiple hops to retrieve the desired content from separate providers. PoolCache performs significantly worse than CaDaCa, despite employing a collaborative caching strategy. This inefficiency primarily stems from its design, which caches both popular and unpopular content indiscriminately. Two key factors contribute to its lower performance compared to CaDaCa and SANR-CMF: first, its reliance on the least recent used (LRU) policy, which frequently evicts and replaces unpopular content; and second, its caching mechanism, which strictly prevents redundancy within any given pool of nodes.

3) Average Route Elongation Factor: Fig. 8(a) and Fig. 8(b) show the relationship between the REF on the y-axis as a function of cache sizes and alpha range on the *x*-axis. Results demonstrate the fact that the SANR-CMF outperforms CaDaCa with various cache sizes and differing alpha values. The superior elongation performance of SANR-CMF is achieved by selecting a group of MSRR and caching these contents at strategically chosen nodes (CCN), thereby increasing data accessibility by placing the requested data near to-end devices or consumers. One more primary reason for SANR-CMF lies in its ability to locally cache all requested content at edge nodes, which are typically located close to end devices. Furthermore, SANR-CMF assigns a designated timespan for every content item at various positions, allowing for the timely discarding of older content to facilitate newly requested content. Lastly, in cases where content designated as MSRR fails to be accommodated due to a lack of available cache space, the least semantically relevant content will be ejected to facilitate the inclusion of new data.

On the contrary, CaDaCa and PoolCache show poorer performance than SANR-CMF in terms of the elongation factor. This subpar performance can be attributed to the additional caching operations that CaDaCa undertakes, such as block 8, block 10, and block 11. When new content reaches, significant ejection operations are required to facilitate unique content names. This is owing to the arrival of unique content that belongs to existing categories on the corresponding node. This cycle recurs with every request or content transmission. Moreover, several parameters, including category probability updates and knowledge extraction, are employed to determine content caching, which further increases system overhead. In CaDaCa, content that is likely to be cached is stored at core nodes where cache hits occur. Consequently, it takes a considerable amount of time to retrieve content close to end devices at the edge node, and the content may become worthless by that time. The elongation factor in PoolCache is further exacerbated by the fixed and inefficient positioning of the caching node. This placement is determined by a hashing technique that assigns content to a specific node (node ID) in the network, without accounting for whether the cached content will be required at that location in the future. In scenarios where a consumer initiates requests and the hash function is suboptimal, leading to content being cached at a location distant from the consumer, this misallocation results in an increase in the REF. Additionally, PoolCache caches unpopular data as well, which reduces the likelihood of popular content being stored at the cache node. Especially at the same time, when the content eviction process is also ongoing. Consequently, requests for popular content often have to traverse multiple nodes to reach the desired destination.

4) Latency: Fig. 9(a) and (b) illustrate the average latency against increasing alpha range and cache size. Results demonstrate that SANR-CMF consistently outperforms state-of-the-art CaDaCa. The superior performance of SANR-CMF can be related to its efficient cache utilization, particularly when content is replaced between nodes. This approach enhances latency performance because its semantic popularity remains intact when content is replaced by a CCN. As a result, requests for corresponding content can be served from the closest node, eliminating the need to forward requests to distant nodes, which reduces latency. The performance of both strategies improves as cache size and alpha values increase. Larger cache sizes allow network nodes to store more content, enabling a greater number of incoming requests to be fulfilled by nearby nodes, which further reduces latency. SANR-CMF excels by strategically selecting and caching the MSRR at optimal locations, such as CCN and corresponding edge nodes, ensuring efficient content delivery to end devices. On the contrary, CaDaCa exhibits poor latency performance. This is the result of CaDaCa categorizing the data into different categories

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.



(a) (b)

Fig. 9. Average latency impact as a function of (a) caching popularity and (b) cache size.

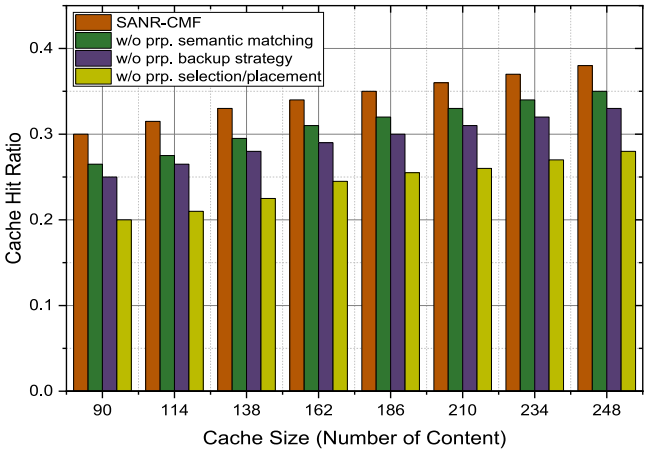


Fig. 10. Ablation studies for SANR-CMF.

(such as sports, games, etc.) at each node and allowing content to be cached at every network node under the corresponding category. When new content arrives, many evictions need to make space, which takes time and often forces requests to be sent to distant nodes for the content. PoolCache demonstrates suboptimal latency performance due to its system design. It stores cached data at fixed positions (cache node IDs). When data is present at the designated cache node ID, it is successfully cached. However, if the data is evicted under the LRU policy due to the arrival of new content or if the data is not found, requests for the content are delayed. This delay occurs because the request must be routed to another pool or cluster to retrieve the data. This design also strictly prevents redundancy within any given pool of nodes. Furthermore, PoolCache also requires the inclusion of an additional name within the namespace field of the packet (for both interest and data packets) in NDN. Specifically, PoolCache utilizes Packet.name1 and Packet.name2 to represent the two names within the packet, whether it is an interest or data packet. This leads to additional computations and checks at each node. As a result, latency is increased.

5) Ablation Studies: We conduct ablation studies to evaluate the effectiveness of each component integrated into SANR-CMF. The cache hit rates for all methods increase as the cache size expands, as illustrated in Fig. 10. In the ablation of SANR-CMF, the DNN-based content-centric placement module is replaced with a random caching strategy. The removal of the DNN-based placement significantly impairs SANR-CMF's ability to predict future content request patterns, resulting in the lowest cache hit rate across all evaluated scenarios. For the SANR-CMF without semantic name matching, the cache hit rate is limited by the performance of the EMP and fails to leverage the semantic information embedded in NDN content names. In the SANR-CMF without the backup strategy, although a DNN-based content-centric model is employed, the CCN discards the MSRR once the cache reaches full capacity. This action facilitates the accommodation of new incoming data but consequently forces subsequent content requests to be redirected to the producer, resulting in a decreased cache hit rate. Therefore, the ablation studies validate the effectiveness of the individual component integrated into SANR-CMF.

6) NLP Classification Performance (NCP): We compared the proposed NLP approach with state-of-the-art INT8-based quantized TinyBERT [37], [38], [39], an efficient model known for the hardware friendly and light weight environment, unlike other NLP models, which consume intensive memory space and high inference time. Quantized TinyBERT consists of four transformer layers, 12 attention heads, and a hidden size 312. The model uses a maximum sequence length of 256, an embedding dimension of 312, and is optimized using Adam. The training setup includes ten epochs, L1 loss with categorical cross-entropy, and a batch size of 32. For the ElasticNet model in the proposed NLP approach, the hyperparameters include alpha range from 0.0001 to 10, l1_ratio set between 0.5 and 0.7, max_iter ranging from 1000 to 5000, tol set at $1e-4$ and $1e-5$, and fit_intercept which is set to *True*.

The results obtained from the NCP evaluation are presented in Table IV. TinyBERT achieves a slightly higher accuracy of 87.17%, albeit with a relatively higher inference time of 55 ms and a model size of 5.2 MB. The increased inference time and larger model size are attributed to the computational overhead of transformer-based operations. In contrast, the proposed approach attains a comparable accuracy of 86.45%,

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
NCP EVALUATION

Par	Acc(%)	Inf.T(ms)	M.Size(mb)
T.BERT	87.17	55	5.2
Pr.Appr	86.45	12	1.3

while significantly reducing the inference time to 12 ms and the model size to 1.3 MB. The reduced inference time and compact model size are primarily due to the absence of transformer-based operations or deep learning techniques. Instead, the proposed NLP approach relies on a lightweight linear model, which ensures computational efficiency while maintaining competitive performance. It is essential to highlight that the accuracy rates attained by both schemes lie within a range deemed highly effective for semantic similarity tasks on the sentence level [40], [41]. An additional advantage of the proposed approach is its superior efficiency in terms of computational speed and memory usage. This makes the proposed method particularly well-suited for scenarios where model size and inference time are of critical importance, such as in resource-constrained environments.

VI. CONCLUSION AND FUTUREWORK

In this article, we present a novel NDN IoT-based approach called SANR-CMF to address naming issues that may arise in the exact-match policy of cache tables. To this end, the proposed scheme enhances the SSR and overall cache performance. SANR-CMF operates through four mechanisms: content naming resolution, an NLP-based approach to interpret the semantics of content names, content selection where a group of MSRR is chosen, content placement where content-centric nodes are designated for caching the selected MSRR, and content replacement that preserves the importance of content while caching content at backup nodes. For performance evaluation, the state-of-the-art strategy, CaDaCa and PoolCache, were selected, using the GEANT topology, and the event-based caching simulator Icarus was employed to compare all approaches. SANR-CMF has demonstrated significant improvements in SSR, cache hit rate, latency, REF, and ablation studies across varying alpha values (0.55, 0.75, and 0.95) and cache sizes, consistently outperforming CaDaCa and PoolCache in terms of caching efficiency. In our future work, we plan to design the Raspberry Pi-based testbed implementation to analyze the practical performance of SANR-CMF. We also aim to explore environment-specific NDN scenarios with varying user behaviors. Additionally, we intend to investigate caching approaches in event-oriented IoT environments, such as Industrial IoT (IIoT) and Internet of Vehicles (IoV). Additionally, we will develop intelligent caching approaches tailored to distinct data types, access patterns, and user behaviors.

REFERENCES

[1] S. Madakam, R. Ramaswamy, and S. Tripathi, "Internet of Things (IoT): A literature review," *J. Comput. Commun.*, vol. 3, no. 5, pp. 164–173, 2015.

[2] B. Nour et al., "A survey of Internet of Things communication using ICN: A use case perspective," *Comput. Commun.*, vols. 142–143, pp. 95–123, Jun. 2019.

[3] W. M. H. Azamuddin, A. H. M. Aman, H. Sallehuddin, K. Abualsaud, and N. Mansor, "The emerging of named data networking: Architecture, application, and technology," *IEEE Access*, vol. 11, pp. 23620–23633, 2023.

[4] D. Saxena, V. Raychoudhury, N. Suri, C. Becker, and J. Cao, "Named data networking: A survey," *Comput. Sci. Rev.*, vol. 19, pp. 15–55, Feb. 2016.

[5] M. A. Hail, "IoT-NDN: An IoT architecture via named data netwoking (NDN)," in *Proc. IEEE Int. Conf. Ind. 4.0, Artif. Intell., Commun. Technol. (IAICT)*, 2019, pp. 74–80.

[6] L. V. Yovita and N. R. Syambas, "Caching on named data network: A survey and future research," *Int. J. Elect. Comput. Eng.*, vol. 8, no. 6, pp. 4456–4466, 2018.

[7] S. Shannigrahi, C. Fan, and C. Partridge, "What's in a name? Naming big science data in named data networking," in *Proc. 7th ACM Conf. Inf.-Centric Netw.*, 2020, pp. 12–23.

[8] I. Abdullahi, S. Arif, and S. Hassan, "Survey on caching approaches in information centric networking," *J. Netw. Comput. Appl.*, vol. 56, pp. 48–59, Oct. 2015.

[9] O. Serhane, K. Yahyaoui, B. Nour, and H. Moungla, "A survey of ICN content naming and in-network caching in 5G and beyond networks," *IEEE Internet Things J.*, vol. 8, no. 6, pp. 4081–4104, Mar. 2021.

[10] G. M. Raza, Z. S. Butt, S. Latif, and A. Wahid, "Sentiment analysis on COVID tweets: An experimental analysis on the impact of count vectorizer and TF-IDF on sentiment predictions using deep learning models," in *Proc. Int. Conf. Digit. Futures Transformative Technol. (ICoDT2)*, 2021, pp. 1–6.

[11] S. Wang, H. Chen, and Y. Wang, "Collaborative caching for energy optimization in content-centric Internet of Things," *IEEE Trans. Comput. Social Syst.*, vol. 9, no. 1, pp. 230–238, Feb. 2022.

[12] Y. Song, H. Ma, and L. Liu, "TCCN: Tag-assisted content centric networking for Internet of Things," in *Proc. IEEE 16th Int. Symp. World Wireless, Mobile Multimedia Netw. (WoWMoM)*, 2015, pp. 1–9.

[13] Y. Meng, M. A. Naeem, R. Ali, and B.-S. Kim, "EHCP: An efficient hybrid content placement strategy in named data network caching," *IEEE Access*, vol. 7, pp. 155601–155611, 2019.

[14] M. Amadeo, C. Campolo, G. Ruggeri, and A. Molinaro, "Beyond edge caching: Freshness and popularity aware IoT data caching via NDN at Internet-scale," *IEEE Trans. Green Commun. Netw.*, vol. 6, no. 1, pp. 352–364, Mar. 2022.

[15] I. Psaras, W. K. Chai, and G. Pavlou, "Probabilistic in-network caching for information-centric networks," in *Proc. 2nd ICN Workshop Inf.-Centric Netw.*, 2012, pp. 55–60.

[16] A. T. Herouala, B. Ziani, C. A. Kerrache, A. E. K. Tahari, N. Lagraa, and S. Mastorakis, "CaDaCa: A new caching strategy in NDN using data categorization," *Multimedia Syst.*, vol. 29, no. 5, pp. 2935–2950, 2023.

[17] B. Alahmri, S. Al-Ahmadi, and A. Belghith, "Efficient pooling and collaborative cache management for NDN/IoT networks," *IEEE Access*, vol. 9, pp. 43228–43240, 2021.

[18] A. Aboodi, T.-C. Wan, and G.-C. Sodhy, "Survey on the incorporation of NDN/CCN in IoT," *IEEE Access*, vol. 7, pp. 71827–71858, 2019.

[19] M. Amadeo, "A literature review on caching transient contents in vehicular named data networking," *Telecom*, vol. 2, no. 1, pp. 75–92, 2021.

[20] O. Hahm, E. Baccelli, T. C. Schmidt, M. Wahlisch, and C. Adjih, "A named data network approach to energy efficiency in IoT," in *Proc. IEEE Globecom Workshops (GC Wkshps)*, 2016, pp. 1–6.

[21] Y. Meng, M. A. Naeem, M. Sohail, A. K. Bashir, R. Ali, and Y. Bin Zikria, "Elastic caching solutions for content dissemination services of ip-based Internet technologies prospective," *Multimedia Tools Appl.*, vol. 80, no. 11, pp. 17023–17023, 2021.

[22] G. Jaber, N. Pastei, F. Rahal, and A. Abboud, "Naming and routing scheme for data content objects in information-centric network," in *Proc. IEEE Open Conf. Elect., Electron. Inf. Sci. (eStream)*, 2020, pp. 1–5.

[23] G. M. Raza, I. Ullah, M. S. Ud Din, M. A. U. Rehman, and B.-S. Kim, "INF-NDN IoT: An intelligent naming and forwarding in name data networking for Internet of Things," *IEEE Access*, vol. 12, pp. 114319–114337, 2024.

[24] T. Mochida et al., "Naming scheme using NLP machine learning method for network weather monitoring system based on ICN," in *Proc. 20th Int. Symp. Wireless Pers. Multimedia Commun. (WPMC)*, 2017, pp. 428–434.

[25] (Carnegie Mellon Univ., Pittsburgh, PA, USA). *The Lemur Project*. (2003). [Online]. Available: http://www.lemurproject.org

[26] S. Bird, E. Klein, and E. Loper, *Natural Language Processing With Python*. Sebastopol, CA, USA: O'Reilly Media, Inc., 2009. [Online]. Available: https://www.nltk.org/

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.

[27] J. P. McCrae, E. Rudnicka, and F. Bond, "English WordNet: A new open-source WordNet for English," *K Lexical News*, vol. 28, no. 1, pp. 37–44, 2020.

[28] J. Korzeniewski, "A modification of the Leacock-Chodorow measure of the semantic relatedness of concepts," *Acta Universitatis Lodziensis. Folia Oeconomica*, vol. 6, no. 351, pp. 97–106, 2020.

[29] T. Mikolov, I. Sutskever, K. Chen, G. S. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 26, 2013, pp. 3111–3119.

[30] F. Amini and G. Hu, "A two-layer feature selection method using genetic algorithm and elastic net," *Expert Syst. Appl.*, vol. 166, Mar. 2021, Art. no. 114072.

[31] M. Thevaraja, A. Rahman, and M. Gabirial, "Recent developments in data science: Comparing linear, ridge and lasso regressions techniques using wine data," in *Proc. Int. Conf. Digit. Image Signal Process.*, 2019, pp. 1–6.

[32] A. Anand, C. Muthukrishnan, A. Akella, and R. Ramjee, "Redundancy in network traffic: Findings and implications," in *Proc. 11th Int. Joint Conf. Meas. Model. Comput. Syst.*, 2009, pp. 37–48.

[33] I. Poese, B. Frank, B. Ager, G. Smaragdakis, and A. Feldmann, "Improving content delivery using provider-aided distance information," in *Proc. IMC*, 2010, pp. 22–34.

[34] N. Hardeniya, J. Perkins, D. Chopra, N. Joshi, and I. Mathur, *Natural Language Processing: Python and NLTK*. Birmingham, U.K.: Packt Publishing Ltd, 2016.

[35] L. Saino, I. Psaras, and G. Pavlou, "Icarus: A caching simulator for information centric networking (ICN)," in *Proc. SimuTools*, vol. 7, 2014, pp. 66–75.

[36] A. Conneau et al. (Stanford University, Stanford, CA, USA). *The Stanford Natural Language Inference (SNLI) Corpus*. (2015). Accessed: Nov. 12, 2024. [Online]. Available: https://nlp.stanford.edu/projects/snli/

[37] N. Wang et al., "Deep compression of pre-trained transformer models," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 35, 2022, pp. 14140–14154.

[38] M. W. U. Rahman et al., "Quantized transformer language model implementations on edge devices," in *Proc. Int. Conf. Mach. Learn. Appl. (ICMLA)*, 2023, pp. 709–716.

[39] D. Zheng, J. Li, Y. Yang, Y. Wang, and P. C.-I. Pang, "MicroBERT: Distilling MoE-based knowledge from BERT into a lighter model," *Appl. Sci.*, vol. 14, no. 14, p. 6171, 2024.

[40] M. Han, X. Zhang, X. Yuan, J. Jiang, W. Yun, and C. Gao, "A survey on the techniques, applications, and performance of short text semantic similarity," *Concurrency Comput. Pract. Exp.*, vol. 33, no. 5, 2021, Art. no. e5971.

[41] P. Sunilkumar and A. P. Shaji, "A survey on semantic similarity," in *Proc. Int. Conf. Adv. Comput., Commun. Control (ICAC3)*, 2019, pp. 1–8.

Ghulam Musa Raza received the B.S. degree in computer sciences (major in intelligent robotics) from Comsats University Islamabad, Islamabad, Pakistan, in 2019, the M.S. degree in computer sciences [major in natural language processing (artificial intelligence)] from School of Electrical Engineering and Computer Science, NUST, Islamabad, Pakistan, in 2021. He is currently pursuing the Ph.D., degree with the Department of Communication and Software Engineering, Graduate School, Hongik University, Sejong, South Korea.

From 2017 to 2019, he worked as a Software Engineer with Snaky Solutions Pvt. Ltd., Sahiwal, Pakistan. He served as a Machine Learning Research Assistant with the TUKL lab, NUST, Islamabad, in 2021. He served as a Lecturer with Alhamd Islamic University, Islamabad, from 2021 to 2022. His major research interests are in the fields of natural language processing, the Internet of Things (IoT), information-centric networking, and named data networking.

Ihsan Ullah received the B.S. degree in computer systems engineering from the University of Engineering and Technology Peshawar, Peshawar, Pakistan, in 2019, and the master's degree in computer engineering with a specialty in computer and wireless networks from the Department of Electrical and Computer Engineering, COMSATS University, Islamabad, Pakistan, in 2021. He is currently pursuing the Ph.D. degree with the Department of Software and Communication Engineering, Hongik University, Sejong, South Korea.

He worked as a Research Assistant with Wireless and Communication Lab, Peshawar, for six months. His research interests include NDN, underwater wireless sensor networks, cloud computing, fog computing, vehicular networks, machine learning, and artificial intelligence.

Muhammad Nadeem Ali received the B.S. and M.S. degrees in electronics engineering from International Islamic University Islamabad, Islamabad, Pakistan, in 2012 and 2016 respectively. He is currently pursuing the Ph.D. degree with Hongik University, Sejong, South Korea.

His B.S. degree in electronics engineering was fully funded by the Ministry of Information and Technology, Government of Pakistan, for four years. The scholarship name was ICT & Research and Development Fund. He is currently a member of Broadband Convergence Network Laboratory with Hongik University. From 2015 to 2021, he was working as a Senior Lecture with the Department of Computer Science, Lahore Garrison University, Lahore, Pakistan, where he supervised many final year projects. His major interest packets are 5G wireless communication system, intelligent transport system and its application, Latency issues in networks, edge computing, and name data network.

Dr. Ali was an Assistant Editor of *Lahore Garrison University Journal of Computer Science Technology* from 2017 to 2021.

Byung-Seo Kim (Senior Member, IEEE) received the B.S. degree in electrical engineering from In-Ha University, In-Chon, South Korea, in 1998, and the M.S. degree in electrical and computer engineering and the Ph.D. degree, under the supervision of Dr. Y. Fang, in electrical and computer engineering from the University of Florida, Gainesville, FL, USA, in 2001 and 2004, respectively.

Between 1997 and 1999, he worked with Motorola Korea Ltd., Paju, South Korea, as a Computer Integrated Manufacturing Engineer in Advanced Technology Research and Development. From January 2005 to August 2007, he worked with Motorola Inc., Schaumburg, IL, USA, as a Senior Software Engineer in Networks and Enterprises. His research focuses on Motorola Inc, designing the protocol and network architecture of wireless broadband mission-critical communications. From 2012 to 2014, he was the Chairman of the Department of Software and Communications Engineering with Hongik University, Sejong, South Korea, where he is currently a Professor. His work has appeared in around 167 publications and 22 patents. His research interest packets include the design and development of efficient wireless/wired networks, including link-adaptable/ cross-layer-based protocols, multiprotocol structures, wireless CCNs/NDNs, mobile edge computing, physical layer design for broadband PLC, and resource allocation algorithms for wireless networks.

Prof. Kim served as the General Chair for General Chair of 3rd IWWCN 2017, and the TPC member for the IEEE VTC 2014-Spring and the EAI FUTURE2016, and ICGHIC 2016, 2019 conferences. He served as a Guest Editor for special issues of the *International Journal of Distributed Sensor Networks* (SAGE), IEEE ACCESS, and the *Journal of the Institute of Electrics and Information Engineers*. He also served as a member of the Sejong-city Construction Review Committee and the Ansan-city Design Advisory Board. He is an Associate Editor of IEEE ACCESS.

Authorized licensed use limited to: BEIJING INSTITUTE OF TECHNOLOGY. Downloaded on February 07,2026 at 01:23:31 UTC from IEEE Xplore. Restrictions apply.